

Professional Certificate Programme in Agentic AI and Applications



Deploying & Monitoring Agentic Systems



Revisit the Learning So Far

Let's take a quick look at the key ideas and concepts you have explored in your learning journey.

- Role of Retrieval-Augmented Generation (RAG) in grounding AI responses
- Importance of retrieval pipelines, embeddings, chunking and vector databases
- Use of contextual embeddings and relevance ranking
- Application of Retrieval-Augmented Generation (RAG) across domains
- Future directions in Retrieval-Augmented Generation (RAG)

Topics Covered

- Deploying & Monitoring Agentic Systems
- Case Study
- Hosting Options for AI Agents
- Demo: API Deployment with FastAPI
- Compare Hosting Trade-offs
- Deploy an Agent
- Monitoring & Observability for Agentic Systems

Deploying & Monitoring Agentic Systems

Moving intelligent agents from development to production environments

From Development to Real-World Execution

The deployment challenge

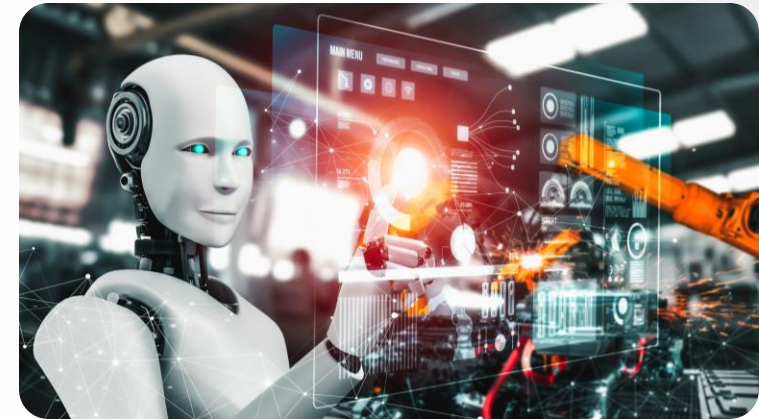
Agents must transition from controlled development environments to live production systems



- **Scalability:** Handles variable user loads
- **Accessibility:** Ensures reliable 24/7 availability
- **Reliability:** Maintains consistent performance under pressure

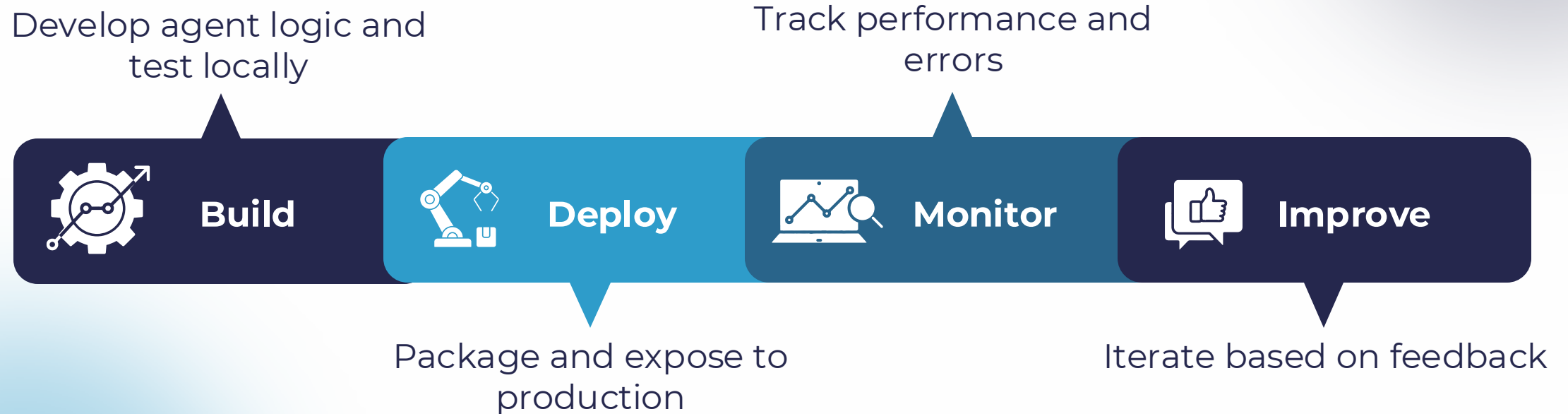
The monitoring imperative

Continuous observation ensures agents deliver value and adapt to evolving conditions



- Performance tracking in real-time
- Error detection and remediation
- Feedback-driven improvements

From Development to Real-World Execution



Session Objectives

Understand practical agent deployments

How real-world systems move from code to production

Compare hosting approaches

Cloud platforms, serverless architectures, embedded systems

Deploy an agent using modern tools

FastAPI, Hugging Face Spaces, Streamlit deployments

Add observability capabilities

LangSmith tracing and OpenTelemetry instrumentation

Reflect on organisational fit

Choose the right deployment model for your context

Why Deployment Matters



Converts prototypes to capability

Transform experimental code into user-facing features that deliver business value



Enables continuous improvement

Use live feedback loops to refine models and evolve features

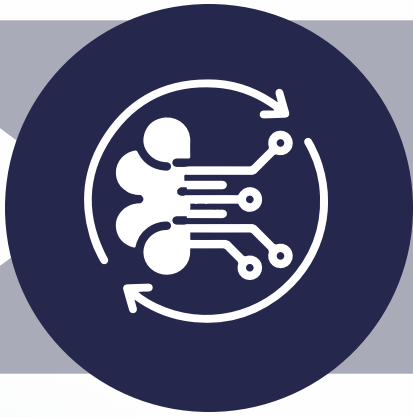


Poor deployment = System failure

Address latency issues, downtime risks and infrastructure costs

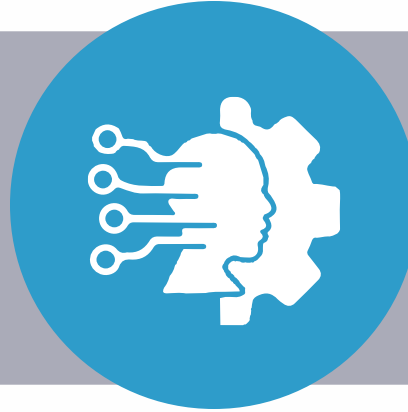
Deployment bridges the gap between what works in a notebook and what works for users

From Sandbox to Production



Local notebooks

Test agent logic during the experimentation phase in controlled environments



APIs

Expose agent capabilities via standardised interfaces



Scalable services

Handle concurrent requests with production-grade systems

From Sandbox to Production

Production requirements

- Continuous monitoring
- Version control
- Security hardening

The Balance

- Responsiveness v. Reliability
- Fast answers must be consistently fast

Case Study

How Agents Are Deployed in Industry

Industry Deployments Overview

Where Agents Run



Real-world agent systems operate across cloud and edge platforms such as:

- Hugging Face Spaces
- AWS SageMaker
- AWS Bedrock
- Azure ML

Industry Deployments Overview



Scalability

Auto-scaling for demand



Observability

End-to-end monitoring



Security

Enterprise-grade protection

Hugging Face Spaces Example

Rapid prototype deployment

Go from code to live demo in minutes

- Simple deployment via web UI
- Git integration for version control
- Support for multiple backends

Supported frameworks

Ideal for prototypes, demos and small-scale agent applications

- FastAPI for REST APIs
- Gradio for interactive UIs
- Streamlit for dashboards

AWS Bedrock and SageMaker Agents



AWS Bedrock

Managed LLM agents with enterprise features

- Auto-scaling infrastructure
- Built-in security controls
- IAM integration



AWS SageMaker

Custom training and serving for specialised models

- Full control over ML pipeline
- Advanced monitoring tools
- Multi-model endpoints

Both platforms integrate seamlessly with CloudWatch for monitoring and IAM for security

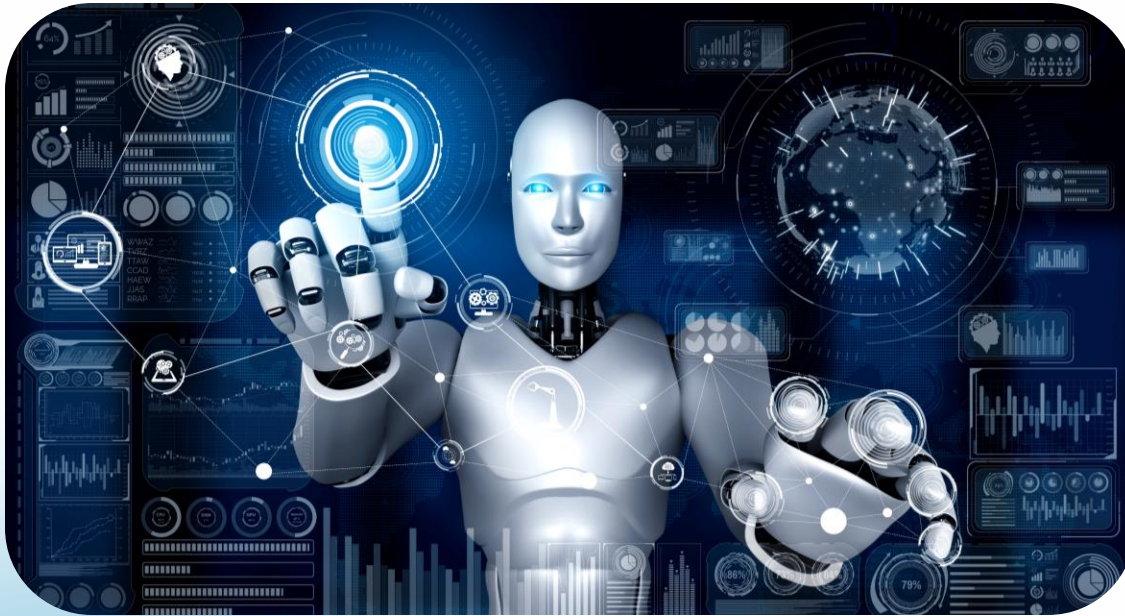
Comparing Industry Approaches

Platform	Strengths	Limitations	Use case
Hugging face	Fast launch, simple UI	Limited custom infrastructure control	Prototypes, demos
AWS Bedrock	Managed security + auto-scaling	Higher cost, vendor lock-in	Enterprise agents
SageMaker	Full customisation, advanced features	Complex setup, steep learning curve	Research + production



Key insight: Choose platforms based on organisational maturity and technical requirements, not just features

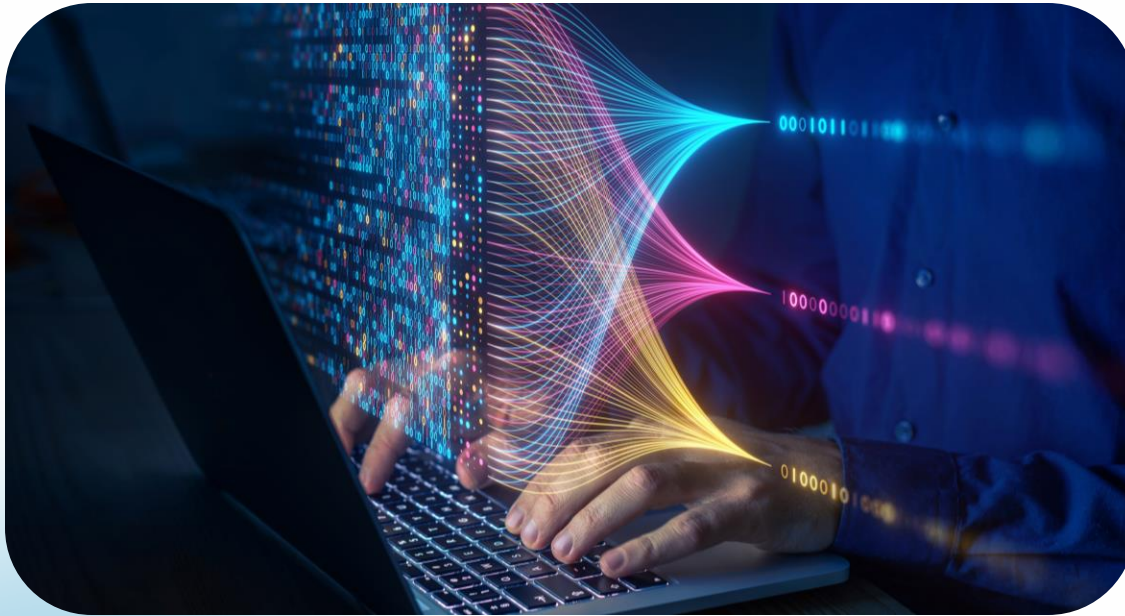
Architecture Pattern



- **Model:** Trained agent logic
- **API:** Interface layer
- **Endpoint:** User-facing service
- **Monitor:** Observability layer

Architecture Pattern

Feedback Loops



Production data flows back to training pipelines,
enabling continuous model improvement

Responsibilities at each stage

- Model: Data science team
- API: Backend engineers
- Endpoint: DevOps team
- Monitor: SRE + ML ops

Deployment Drivers

Latency

Inference speed under load

Responding quickly when users flood the system

Security

Data handling & access controls

Protecting sensitive information and enforcing authentication

Scalability

Auto-scaling for demand spikes

Handling 10x traffic without manual intervention

Cost

Pay-per-use vs always-on instances

Optimising infrastructure spend based on usage patterns

Monitoring in Practice

The Observability Stack



Combined, these tools provide end-to-end visibility from user request to model output

Industry-standard tools for agent monitoring include:

LangSmith

Trace and feedback analysis for LLM interactions

OpenTelemetry

Logs and metrics from distributed services

AgentOps vs LangSmith vs OpenTelemetry

Tool	Focus	Strength	Example Use
AgentOps	End-to-end agent runs	Session analytics	QA bots
LangSmith	Trace and feedback	Prompt optimisation	Chat agents
OpenTelemetry	Logs + metrics	Infrastructure monitoring	APIs

Pro Tip: Use multiple tools for comprehensive observability where they complement rather than compete

Business Impact Metrics

35%

Latency reduction

Via serverless
deployments and edge
caching

25%

Cost Savings

Using auto-scaling and
spot instances

40%

Incident Reduction

Through automation that
lowers manual handling

Data from industry surveys of production AI
deployments, 2023-2024

Case Study Summary

Deployment models affect outcomes



Speed, cost and control vary dramatically

Observability is non-negotiable



Quality cannot be maintained without visibility

Industry embraces managed platforms



Feedback loops enable continuous adaptation

Hosting Options for AI Agents

Cloud, Serverless and Embedded environments: Choosing the right foundation for your AI infrastructure

Three Core Hosting Paradigms



Cloud hosting:

Managed infrastructure offering enterprise-grade scalability



Serverless:

Event-driven execution with pay-per-use pricing model



Embedded/Edge:

Local deployment for ultra-low latency and offline capability

- Each architecture impacts scalability, latency and operational cost.
- Selection driven by business constraints and use-case requirements.

Cloud Hosting: Enterprise-Grade Infrastructure



Comprehensive infrastructure with built-in monitoring, security and compliance tools.

Full-service managed platforms are

AWS: SageMaker, Bedrock

Google Cloud: Vertex AI

Azure: OpenAI Service

Best for: Enterprise agents requiring high uptime, deep integration and regulatory compliance.

Cloud Hosting Advantages



Elastic scaling

Adjust traffic patterns automatically through load balancing without manual intervention



Native integration

Connect seamlessly to monitoring, logging, security services and data warehouses



Multi-agent orchestration

Optimise complex workflows with multiple coordinated agents and data-intensive processing

Serverless: Event-Driven Efficiency

Trigger event

API call, webhook or
scheduled timer

Execute function

Code runs in isolated
container

Return response

Resources deallocate
immediately

AWS Lambda

Industry standard with
extensive ecosystem

Vercel functions

Optimised for edge
deployment

Cloudflare workers

Global distribution,
minimal cold starts

Serverless Trade-offs

Advantages	Constraints
Automatic scaling to zero	Cold start latency (100-3000ms)
No infrastructure management	Limited runtime environments
Pay only for execution time	Execution time limits (5-15 min)
Instant deployment pipelines	Reduced observability control

Embedded & Edge Deployment

Local processing



Agents run directly on IoT devices, wearables or industrial hardware

Ultra-low latency



Agents deliver sub-millisecond response times without network round-trips

Offline resilience



Agents continue functioning during network outages or connectivity loss

Common applications: Industrial automation, autonomous robotics, medical devices, smart manufacturing

Hosting Model Comparison

Hosting model	Latency	Cost	Control	Maintenance
Cloud	Medium	High	High	Medium
Serverless	High (cold starts)	Low	Medium	Low
Embedded	Very Low	Medium	High	High

Security & Governance Framework

Credential management

Secure storage of API keys using vaults (AWS Secrets Manager, HashiCorp Vault)

Network isolation

Deploy within VPC with private subnets and security groups

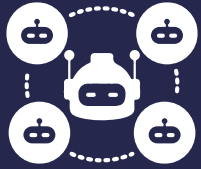
Data compliance

Compliance with PII, GDPR and data residency requirements

Access control

Role-based access control for agent interactions and admin functions

Production Deployment Checklist



Environment selection

Evaluate cloud, serverless, or embedded based on latency and scale requirements



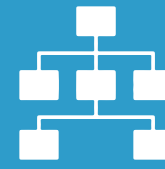
Containerisation

Package agent with Docker for consistent deployment across environments



Observability stack

Implement structured logging and telemetry with OpenTelemetry



LLM tracing

Integrate LangSmith for debugging prompt chains and model interactions



Monitoring & alerts

Configure dashboards (Grafana, Datadog) with threshold-based alerting

Common Scaling Pitfalls



Resource over-provisioning

Excessive capacity allocation leads to unnecessary cost spikes monitor utilisation closely



Rate limit blindness

Ignoring API quotas and timeout thresholds causes cascading failures



Security gaps

Auto-scaling without updated security policies creates vulnerabilities



Cold start neglect

Serverless latency impacts user experience implement warming strategies

Choosing Your Hosting Strategy

Cloud



When to choose: Need enterprise control, deep integrations, compliance requirements

Serverless



When to choose: Event-driven workloads, cost optimisation, rapid iteration

Embedded



When to choose: Latency-critical applications, offline operation, edge processing

Hybrid Hosting Architectures

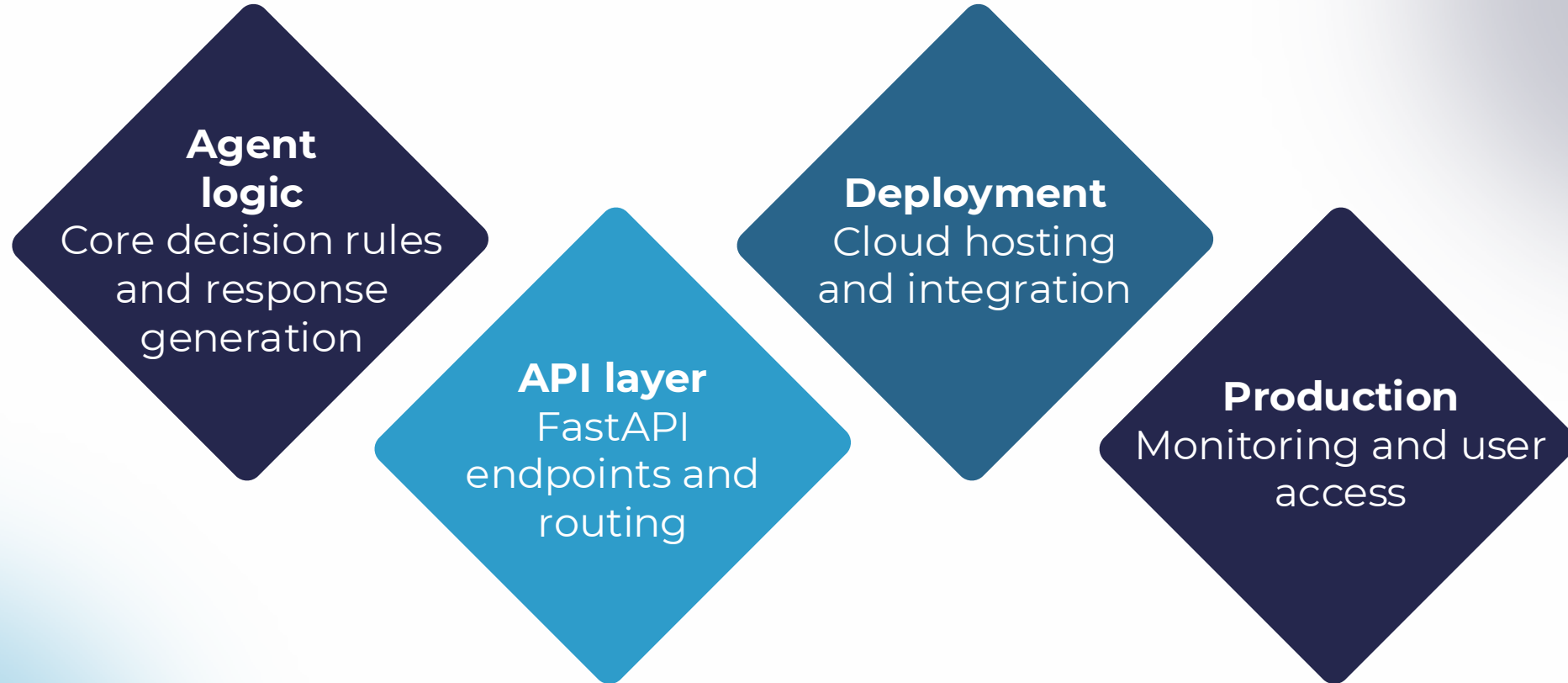


- Hybrid architectures combine models for optimal performance
- Example: Cloud-based orchestration with edge caching for latency-sensitive endpoints
- Align architecture choice with workload, budget and capabilities

Demo: API Deployment with FastAPI

Building production-ready agent APIs with FastAPI—from setup to deployment and monitoring integration

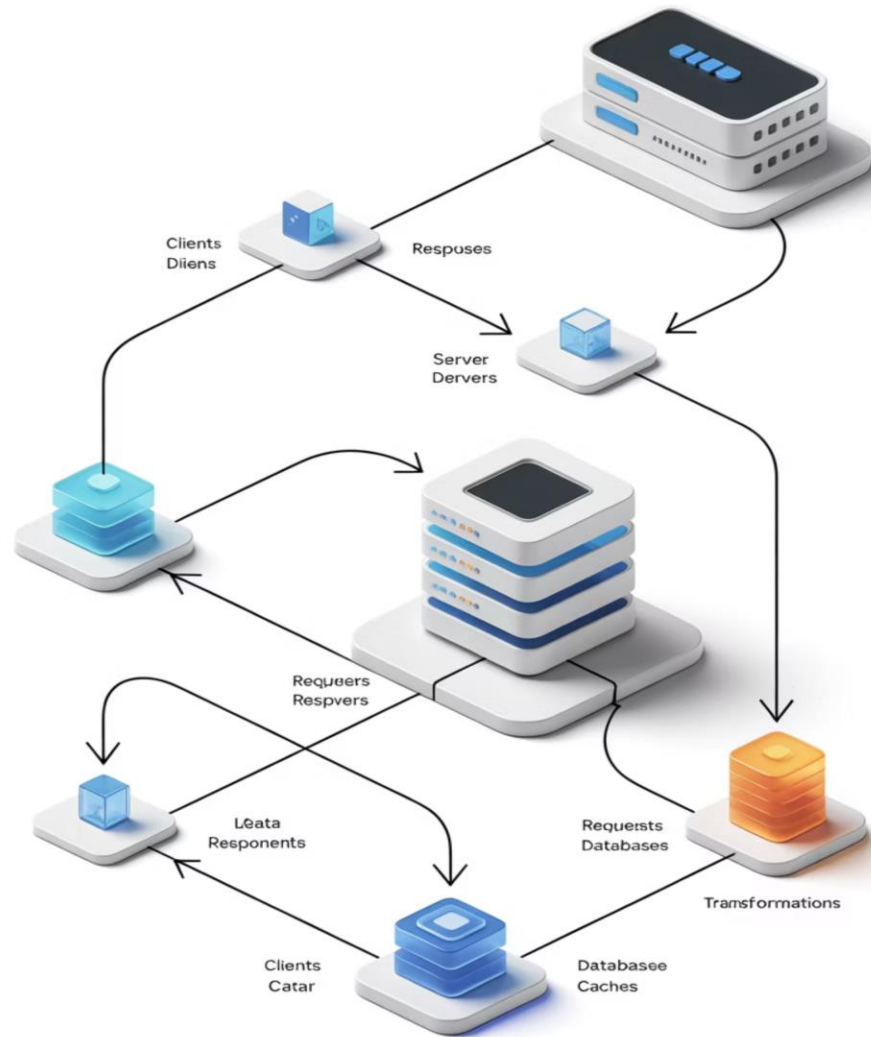
Demo Overview



Transform agent logic into accessible REST endpoints—bridging development to production environments

Demo Overview

REST API Endpoint Architecture



Why FastAPI for Agent Deployment?

Performance Benefits



Delivers a
lightweight, high-
performance web
framework



Enables async
processing for LLM
workloads



Generates API
documentation
automatically



Supports modern
Python type hints
natively

Why FastAPI for Agent Deployment?

HTTP endpoints



Serves agents via REST APIs

Async ready



Handles concurrent requests efficiently

Easy integration



Connects monitoring tools seamlessly

Key Components of API Deployment

Agent Logic

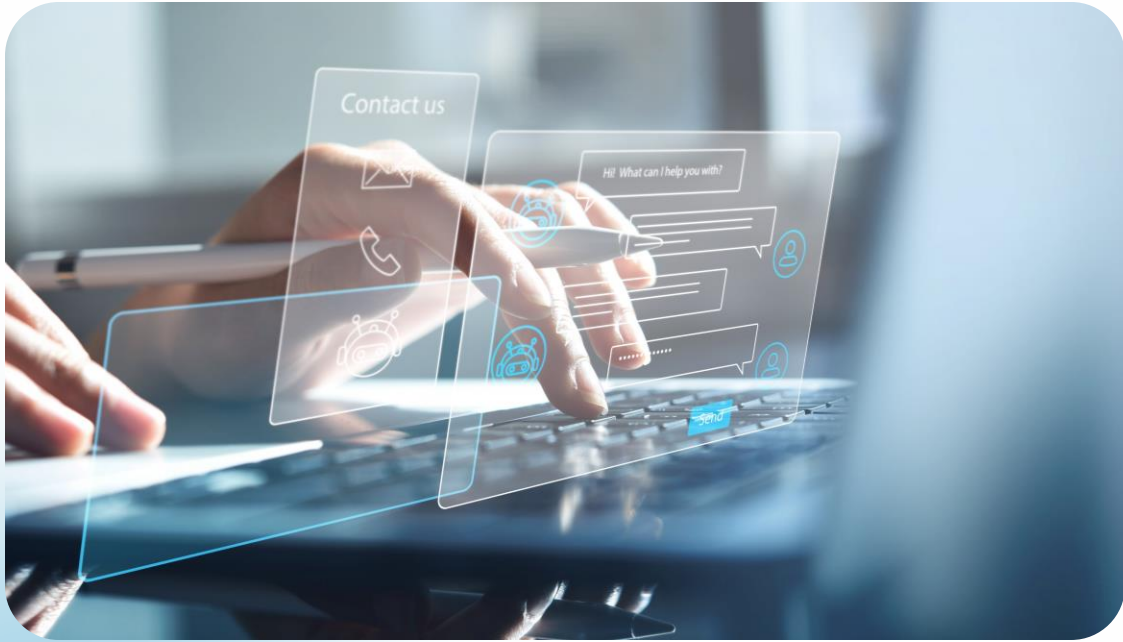


- LLM model loading and initialisation
- Prompt engineering and context management

Develops the response generation engine and decision rules

Key Components of API Deployment

API Routes



Define endpoint specifications for
input/output

- Request validation and parsing
- Response formatting and serialisation

Key Components of API Deployment

Model Management



- Efficient model initialisation strategies
- Memory optimisation techniques

Handles loading, caching and version control

Observability



- Health checks and status reporting
- Trace collection and metrics export

Deployment Flow: Dev to Production



Define endpoints

Create FastAPI routes for agent input/output handling



Local testing

Run locally and test with Postman, curl, or Python clients



Cloud deployment

Deploy to Hugging Face Spaces, Streamlit Cloud or container platforms



Monitor & iterate

Integrate LangSmith and OpenTelemetry for production observability

Testing the API

Validation Checklist

	Send sample requests to endpoints
	Verify response format and accuracy
	Measure latency and throughput
	Check error handling behaviour
	Enable response logging

Testing the API

HTTP endpoints



Input validation

Ensure proper request schema compliance

Async ready



Performance metrics

Track latency, token usage and response time

Easy integration



Log collection

Capture traces for monitoring integration

Integration & Hosting Options



Hugging face spaces

Quick deployment for public demos with built-in GPU support



Streamlit Cloud

Interactive UI with seamless backend API integration



LangSmith integration

Add callbacks for trace visualisation and debugging

Choose hosting platforms based on your use case—from rapid prototyping to production-scale deployment

Security & Deployment Best Practices



Authentication

Protect endpoints with API keys or OAuth 2.0



Secrets management

Use environment variables for all sensitive data



Rate limiting

Add throttling to prevent abuse and control costs



Error handling

Implement graceful failures and retry logic

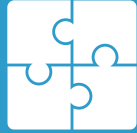
Critical: Deploy only in HTTPS environments to ensure data encryption in transit

Demo Summary



Develop

Build agent logic with
FastAPI endpoints



Deploy

Host on cloud
platforms with
proper
security



Monitor

Integrate observability
tools for production
visibility



Improve

Iterate based on
performance
insights

Compare Hosting Trade-offs

Explore deployment models to find the right fit for your AI agent. Understanding trade-offs across cloud, serverless and embedded hosting is essential for optimal performance.

Activity Overview

Cloud hosting



Managed infrastructure with powerful compute resources and seamless integration capabilities

Serverless



Pay-per-use model with automatic scaling and minimal infrastructure management

Embedded



Edge deployment with complete control and zero latency for IoT applications



Compare models based on cost, latency and maintenance to identify optimal deployment strategies for specific agent requirements.

Activity Instructions



Read

Review each deployment scenario carefully



Evaluate

Assess based on cost, latency, maintenance needs



Record

Capture key insights and reasoning on worksheet



Discuss your choices with peers to explore different perspectives and trade-off considerations.

Deployment Scenarios

Which Hosting Model Fit Best

IoT edge agent



Monitors factory sensors in real-time with millisecond response requirements

Global chatbot



Serves millions of users worldwide with variable traffic patterns

Research prototype



Requires rapid iteration and frequent model updates during development

Evaluation Matrix

Document Your Analysis For Each Scenario Using Framework

Use case	Preferred model	Key benefit	Risk
IoT Edge Agent			
Global Chatbot			
Research Prototype			

Fill in during activity. Consider performance requirements, budget constraints and operational complexity.

Deploy an Agent

Transform theory into practice with hands-on deployment experience using modern cloud platforms.

Exercise Overview



Deploy

Launch a working agent using Hugging Face Spaces or Streamlit Cloud



Validate

Test deployment through live interaction and verify functionality



Monitor

Observe latency, UI responsiveness and backend logs for insights

Gain practical experience with deployment workflows and performance monitoring techniques.

Environment Setup



- **Platform access:** Create or log in to Hugging Face Spaces or Streamlit Cloud account
- **Project files:** Upload `main.py` or `app.py` containing your agent logic
- **Dependencies:** Configure `requirements.txt` with all necessary packages and versions

Ensure all dependencies are pinned to specific versions to avoid deployment failures.

Deployment Steps

Create new Space or App
in your chosen platform

**Launch
workspace**

Connect logic

Upload and link your
agent logic file to the
runtime

Run initial test and verify
app loads correctly

Test preview

Share deployment

Generate public link or
embed code for
distribution

Testing the Deployment

Validation Checklist

	Send sample inputs through web UI
	Measure output latency and accuracy
	Verify backend logs show clean execution
	Test edge cases and error handling
	Confirm expected behaviour across inputs

Document response times and any unexpected behaviours for optimisation insights.

Troubleshooting Tips



App won't start

- Check dependencies match runtime environment
- Verify Python version compatibility in requirements



Blank page loads

- Confirm port and endpoint mapping
- Review application logs for binding errors



Slow model loading

- Implement caching or preloading
- Consider using quantised models for faster startup

Reflection Questions

“



- Was your chosen platform easy to deploy on?
- How did performance differ between local and hosted environments?
- What improvements could enhance reliability or speed?

”



Share insights with your group to compare platform experiences and identify best practices.

Exercise Summary

Deploy

Validates agent readiness for production use

Monitor

Builds intuition through log analysis and metrics

Optimise

Sets foundation for scaling and performance tuning

Deployment experience is essential preparation for production AI agent management and observability practices.

Demo: Deploying and Testing an Agentic API with FastAPI

Monitoring & Observability for Agentic Systems

Building production-ready agents through
comprehensive monitoring strategies

Why Monitoring Matters

Beyond deployment

Deployment marks the beginning, not the end

- Agents evolve and adapt post-launch
- Continuous reliability validation required
- User behaviour drives system refinement

Critical detection points

Critical detection points

- Model drift identification
- Latency threshold violations
- User dissatisfaction signals
- Accuracy degradation patterns

Why Monitoring Matters



- **Deploy:** Release to production
- **Monitor:** Track performance
- **Improve:** Optimise and iterate

What to Monitor in Agentic Systems



Performance metrics

- Response latency
- System throughput
- Uptime percentage
- Resource utilisation



Quality indicators

- Response accuracy
- Output coherence
- Content relevance
- Task completion rate



Behavioural signals

- Agent drift detection
- Hallucination frequency
- Bias manifestation
- Decision consistency



Usage Analytics

- Active session counts
- User engagement depth
- Feature adoption rates
- Retention patterns

Observability Stack Overview

Comprehensive Observability Across Systems

LangSmith layer

LLM-specific tracing and prompt analysis

- Prompt tracking
- Execution traces
- User feedback capture

OpenTelemetry layer

Infrastructure-level observability

- Distributed logs
- Performance metrics
- API-level traces

Together, LangSmith and OpenTelemetry provide end-to-end visibility from prompt to production

LangSmith Overview

Core Capabilities

Purpose-built for LLM monitoring:

- Complete LLM call capture
- Metadata enrichment
- Visual trace exploration

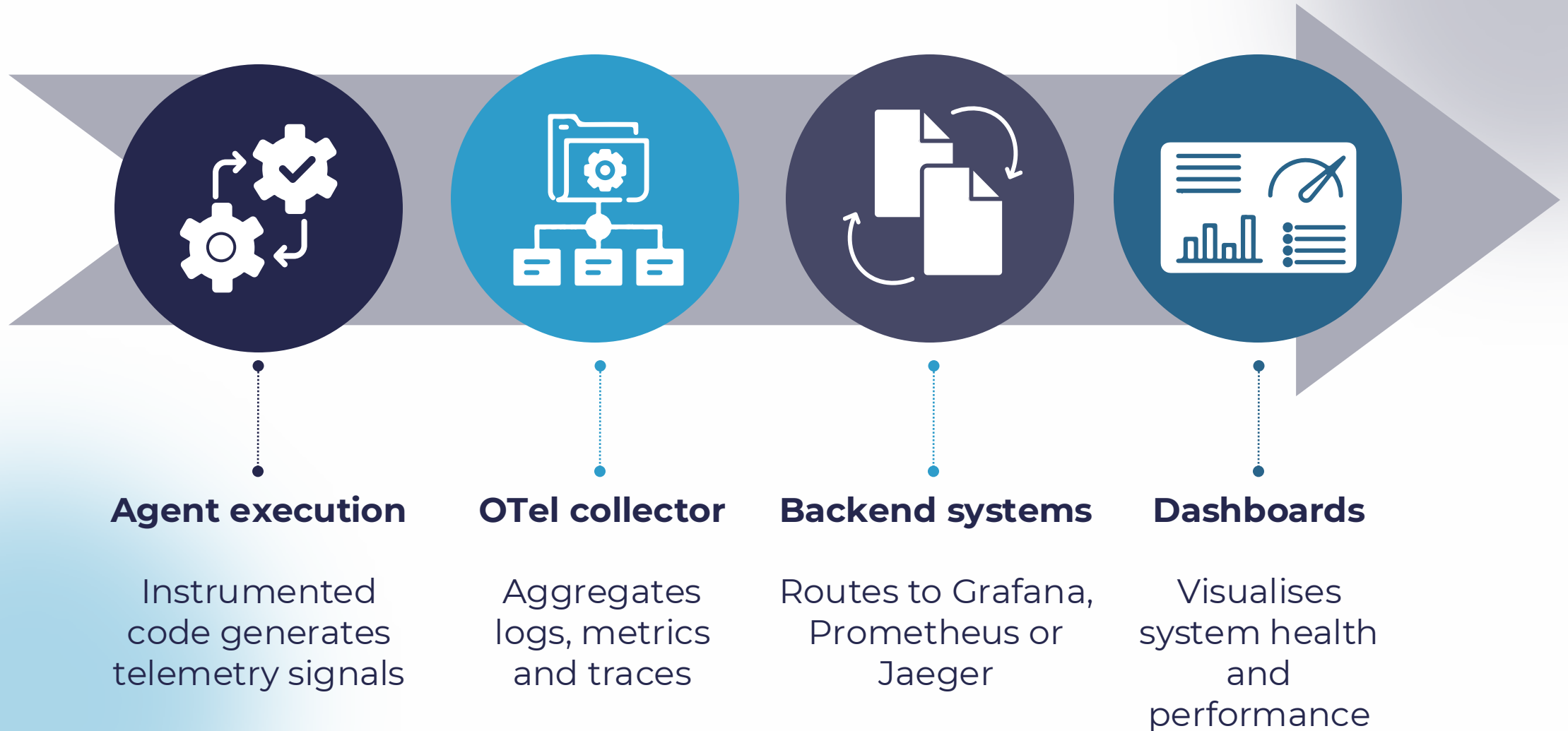
Key benefits

- Debugs complex chains
- Evaluates response quality
- Optimises prompt design
- Tracks model behaviour



Pro Tip: Use LangSmith's trace visualisation to identify bottlenecks in multi-step agent workflows

OpenTelemetry Integration



OpenTelemetry Integration

Unified framework



Language-agnostic, vendor-neutral standard for observability across your entire stack

Distributed tracing



Track requests across microservices, APIs and agent pipeline components

Mini Case: Detecting Agent Drift



Scenario: Customer support agent begins providing outdated product information

Mini Case: Detecting Agent Drift



Problem detection

Identifies incorrect policy details introduced after recent product updates



LangSmith investigation

Reveals response deviations caused by deprecated knowledge base entries



OpenTelemetry analysis

Detects 300 ms latency spikes from external API dependencies leading to stale cache fallbacks



Resolution

Implements knowledge base refresh and API timeout adjustment to eliminate drift

Layered observability reveals root causes invisible to single-tool monitoring

Adding Feedback Loops

Signal Collection Strategy

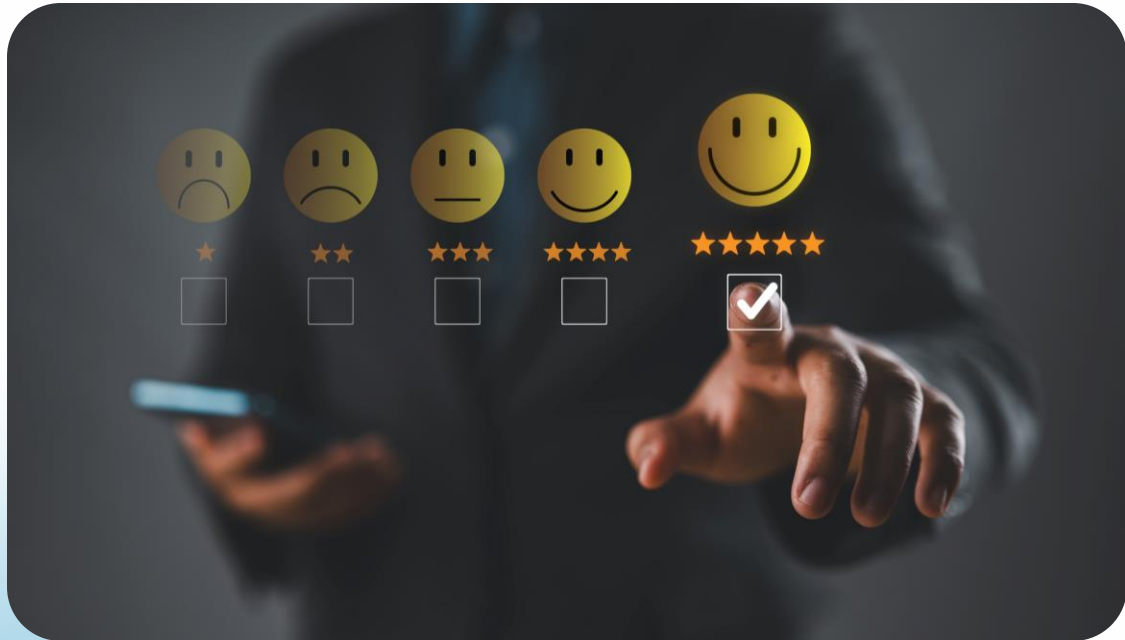
Explicit feedback

- ✓ Thumbs up/down ratings
- ✓ Star-based quality scores
- ✓ Written comments
- ✓ Issue flagging

Implicit signals

- ✓ Session duration
- ✓ Drop-off points
- ✓ Retry frequency
- ✓ Follow-up questions

Adding Feedback Loops



1. **Collect:** Gather user signals
2. **Analyse:** Extract patterns
3. **Update:** Adjust rewards
4. **Retrain:** Optimise behaviour

Building a Monitoring Dashboard



Response time

Track P50, P95, P99 latencies across agent interactions, identifying slow queries



Accuracy metrics

Monitor task completion rates and correctness scores from automated evaluations



Cost per call

Calculate token usage, API costs and infrastructure spend per agent interaction

Building a Monitoring Dashboard

Integration Points



LangSmith

Prompt-level insights
and trace data

Grafana/OTel

System metrics and
alerting

Automation

Anomaly detection with
threshold alerts

Continuous Improvement Workflow



Observe

Collect telemetry
and user feedback



Diagnose

Analyse patterns
and identify issues



Redeploy

Release
improvements to
production



Update

Refine prompts,
retrain models,
adjust rules



Data-driven iteration transforms static agents into adaptive, self-improving systems aligned with reinforcement learning principles

Observability Best Practices

Comprehensive logging

Captures both success and failure cases — errors reveal system weaknesses and improvement opportunities

Links telemetry data with user context, session IDs and request metadata for effective debugging

Contextual correlation

Regular retraining

Schedules periodic fine-tuning based on collected feedback and performance metrics

Anonymises PII, enforce retention policies and maintain audit trails while logging

Privacy & compliance

Remember: Observability without action is surveillance — use insights to drive meaningful improvements

Demo

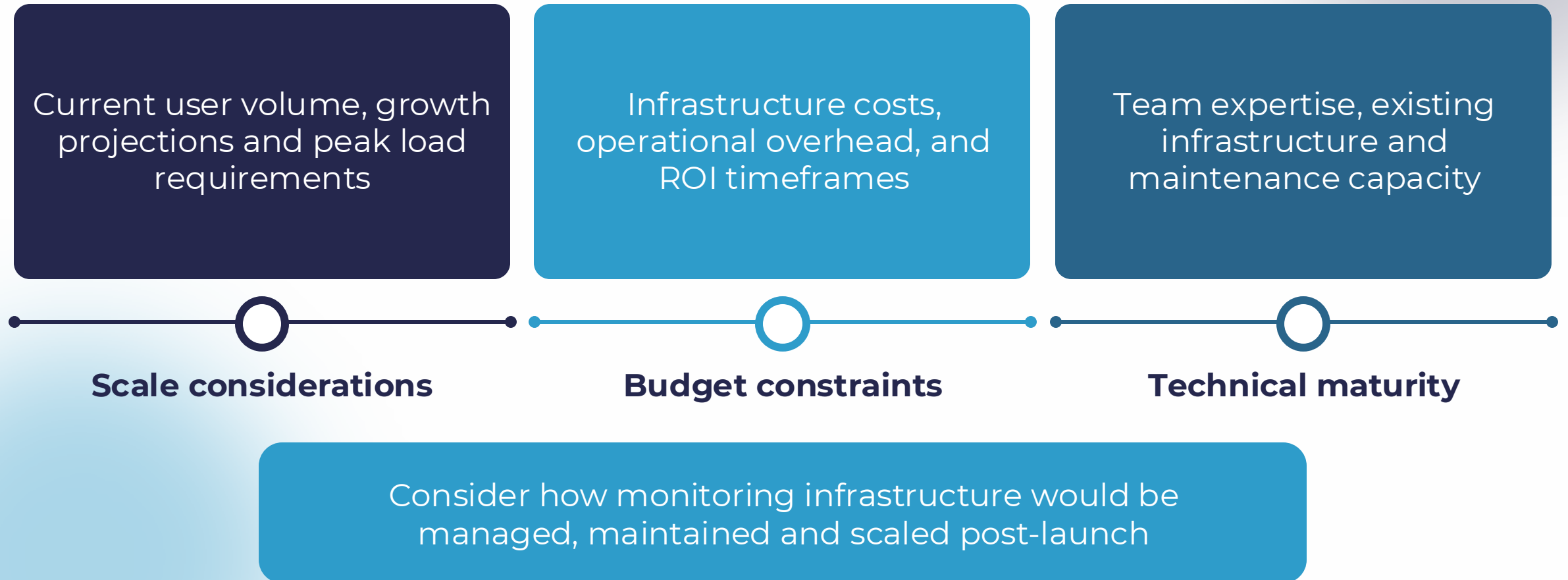
**Monitoring & Observability of
LangChain Agents with Evidently,
LangSmith & OpenTelemetry**

Reflection

Taking time to assess your organisation's readiness
and strategic priorities

Reflection Prompt

Which Deployment Model Fits My Organisation Best



Guiding Questions

Priority trade-offs



What would your team prioritise: cost efficiency, infrastructure control or response latency?

Value assessment



Where could LangSmith + OpenTelemetry integration deliver the most value to your operations?

Current state comparison



How does this observability approach compare to your existing monitoring infrastructure?

Self-Assessment Framework

Assess Current State And Define Next Steps

Dimension	Current readiness	Next step
Hosting infrastructure	Assess existing capabilities	Define deployment strategy
Monitoring setup	Inventory current tools	Plan observability integration
Feedback integration	Review feedback mechanisms	Design collection pipeline
Team skills	Evaluate technical expertise	Identify training needs
Compliance posture	Audit privacy requirements	Implement data governance

Q & A

Thank you